

Stephen Yang—2476005

Jason Hu—2438998

Jun 8, 2026

Google Dino Game

Overview:

This project is a hardware implementation of the Google Chrome Dino Game on the DE1-SoC FPGA using SystemVerilog. The game is rendered entirely through VGA and recreates the core gameplay of the original Chrome Dino, where players control the dinosaur to avoid incoming obstacles.

The system is organized into several modular hardware components. The **Dino Controller** manages the dinosaur's movement and finite-state machine, including running, jumping, ducking, and game-over states. The **Obstacle Controller** is responsible for spawning, moving, and removing obstacles, while the **Hit Box Detector** performs real-time collision detection between the dinosaur and obstacles. The **Graphic Driver** renders the game scene by reading sprite data from ROM and generating pixel data for the **VGA Driver**, which outputs the video signal to the display.

The final implementation expands upon the original design by supporting both **jump** and **duck** controls through an NES controller, allowing the player to avoid both cactus and flying pterosaur obstacles. An **8-bit LFSR** is used to generate pseudo-random obstacle patterns, creating varied gameplay. A standalone **scoreboard module** increments the player's score over time using the 50 MHz system clock and displays it on the FPGA's seven-segment displays. When a collision is detected, the game enters a **DEAD** state, freezing all gameplay while displaying a dedicated game-over dinosaur sprite.

The project demonstrates FPGA-based game development using VGA graphics, ROM-based sprite rendering, finite-state machine design, collision detection, pseudo-random obstacle generation, and modular hardware architecture.

Video Introduction

 [lab6_video_stephy28_shikunhu](#)

Design Procedure:

TOP-LEVEL BLOCK DIAGRAM

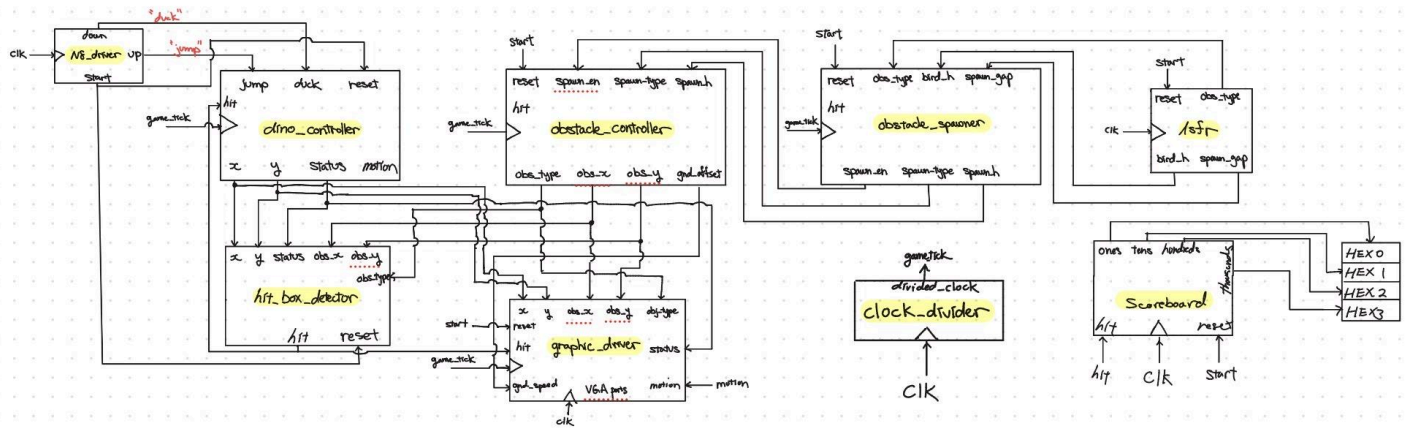


Figure: Top-level Block Diagram

DINO GAME ARCHITECTURE

DEI-SoC

L graphic_driver

L vga_driver

L rendering → given current (x,y) scan position, decide what RGB to output (e.g. sky, dino, obs...)

L dino_sprite → store dino frames in RAM (run_0, run_1, jump, duck), output 1'b pixel given (r,c) & frame

L cactus_sprite → store bitmaps in RAM, output 1'b pixel given local (row,col) and frame

L bird_sprite → store bird frames in RAM (flap_0, flap_1), output 1'b pixel given local (row,col) and frame

L ground_sprite → scrolling ground strip RAM, offsets based on a "scroll counter"

L dino_controller → tracks Dino state (run, jump, duck), handles jump, outputs dino @ top-left

L jump_physics → compute Dino's y-coordinate each frame using velocity + gravity constants, outputs curr y and boolean in_air

L obstacle_controller → Manages obstacles spawning at right time, scrolling, despawn

L obstacle_spawner → Decides when & what to spawn

L lfr → generate "randomness"

L hitbox_detection → output "hit" signal

L scoreboard → hex display

L clk_divider → 50M Hz → slow

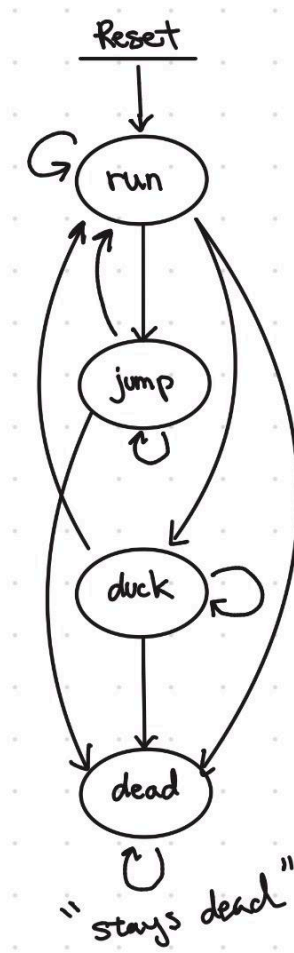


Figure: State Outline of dino_controller on how movement states relate to one another

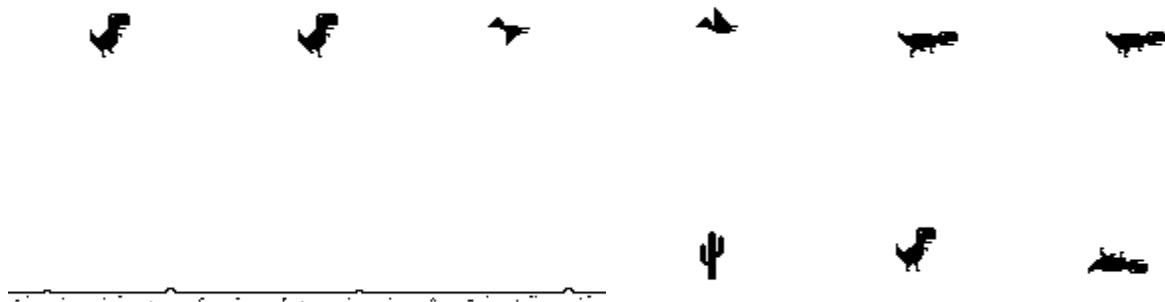


Figure: Dino Sprites, Bird Sprites, Cactus Sprites, and Ground Strip

GAME OVER



Figure: Game Over Frame (but the game won't have running scoreboard on VGA)

Game Logic & Modules

Hit_Box_detector:

Hit_Box_detector is the module that is used to detect if the dino is hit by and of the obstacles. It takes dino's x and y and its status (0 for running, 1 for jumping and 2 for ducking), and a 2 d array for the obstacle's x,y and its type. Each of them represents one obstype (0 for nothing, 1 for cactus and 2 for pterosaur), x and y. The hit box uses the math formula to find if hit or not. For example, the $dino_x + dino_x > obs_x - obs_w$ means there is a right overlap between the dino and the obstacles. So the controller will check both of the obs, if it is 0 which means there are no obs then the hit will always be low, on the other hand if there is an overlap of obstacle and the dino, the hit will be high. The local parameter is coded for adjustable hit box size for the purpose of the game debugging and configuration.

Graphic_driver:

The graphic_drive also takes the data from the game, and x, y and outputs the VGA signal. It takes dino's x and y and its status (0 for running, 1 for jumping and 2 for ducking), and a 2 d array for the obstacle's x,y and its type (0 for nothing, 1 for cactus and 2 for pterosaur). At the same time it takes motion for dino source and pterosaur. Since the running and flapping wing takes 2 images the motion signal decides which signal to use. It also takes a ground speed which is how fast the ground moves and hits a signal to decide that the game is over. IT will output the corresponding VGA signal for display.

The rendering logic is that inside there is a VGA driver, the VGA driver will output the x,y that it needs for its RGB color. The rendering module will decide what RGB is for x,and y. The rendering module will first check if px and py are in the region of any sprite (dino, cactus and pterosaur) based on their x,y. It will also check if it is inside of any ground. The sprite data is stored inside a ROM. The rendering will compare the coordinate to the ROM and output the color of the corresponding pixel. If it hits, it will also check if it is in game over sprite as well. Then it will give color to the VGA driver to output.

Dino_Controller:

This module is a FSM that handles the game logic for Dino such as movements and the states. The only movements in this game are "up" and "down" from the N8 Controller. It takes in inputs game_tick (a slowed down clock), reset (from "Start" button on N8), up, down, and hit (alerts if the Dino collides with an obstacle). The module will constantly be outputting Dino's x & y coordinates and its current status, which is whether it is running, jumping, ducking, or dead. The module keeps track of a lot of local parameters to clarify hard-coded variables such as ground's y-position and the current animation frame,

etc. This helps set the values of the variables constant throughout the module and improves readability. The state of the Dino refers to the motion that it is currently doing.

- no button's pressed: the Dino will be running
- up is pressed: state goes from running to jumping, and then return to running once Dino reaches the ground
- Down is pressed: state goes from running to ducking, and it stays at ducking until button is released

The physics of the jump was a challenge because of so many situations that we need to think about, including velocity of the jump, when can the Dino jump, and how does the game know if Dino's reached the ground again. This module handles all of those scenarios by constantly updating Dino's position and keeping track of contact between ground, Dino, Dino's states, and obstacles.

Obstacle_Controller:

This is the overarching module that handles the spawning, despawning, and scrolling/offset of the obstacles (including ground). It contains "sub-modules" Obstacle_Spawner which feeds it inputs such as spawn_en (an enabler for spawner), spawn_type (2-bit that tells which obstacle is spawning), and spawn_h (height of bird to spawn). The input "hit" signal dictates when the game is over, which will result in the spawner and scrolling motion to freeze entirely. For the ground strip, it increments until it reaches the end of the screen (which is the left-hand side) and it will wrap around to keep producing its pixels from the right-hand side. For the obstacles (cactus and bird/pterosaur), it will spawn in from the right-hand side of the screen and once it reaches the left-hand side, it "despawns" by setting the type of the obstacle to "none", essentially removing it. We also set a maximum limit of only two obstacles allowed on the screen at once to avoid overcrowding the playfield, potentially resulting in the Dino not being able to jump over the obstacles.

Obstacle_Spawner:

The Obstacle_Spawner module is a subset of Obstacle_Controller and it is responsible for taking the pseudo-random selection of obstacle type, obstacle spawn height, and obstacle spawn gap from the LSFR module and passing it through logic that tells the controller when it's allowed to spawn and where to spawn the obstacles. It contains logic that creates a gap between each obstacle so the playfield doesn't overcrowd

LSFR:

The LSFR module is under the Obstacle_Spawner module that creates "randomness" for the obstacle_spawner to decide what and where to spawn. We use an 8-bit LSFR, XOR-ing at the positions 8, 6, 5, 4 to create the "most random" numbers we can get out of 8 bits. The registers at indices 7, 5, 4, 3 will be XOR-ed and then shifted. Then bit 1-0 will be used for selecting obstacle type (the reason we chose 2 bits is so that we can bias an obstacle over another if we want to), bit 2 will be used for selecting bird's height (either high or low), bit 7-3 will be used to select the gap between each spawn.

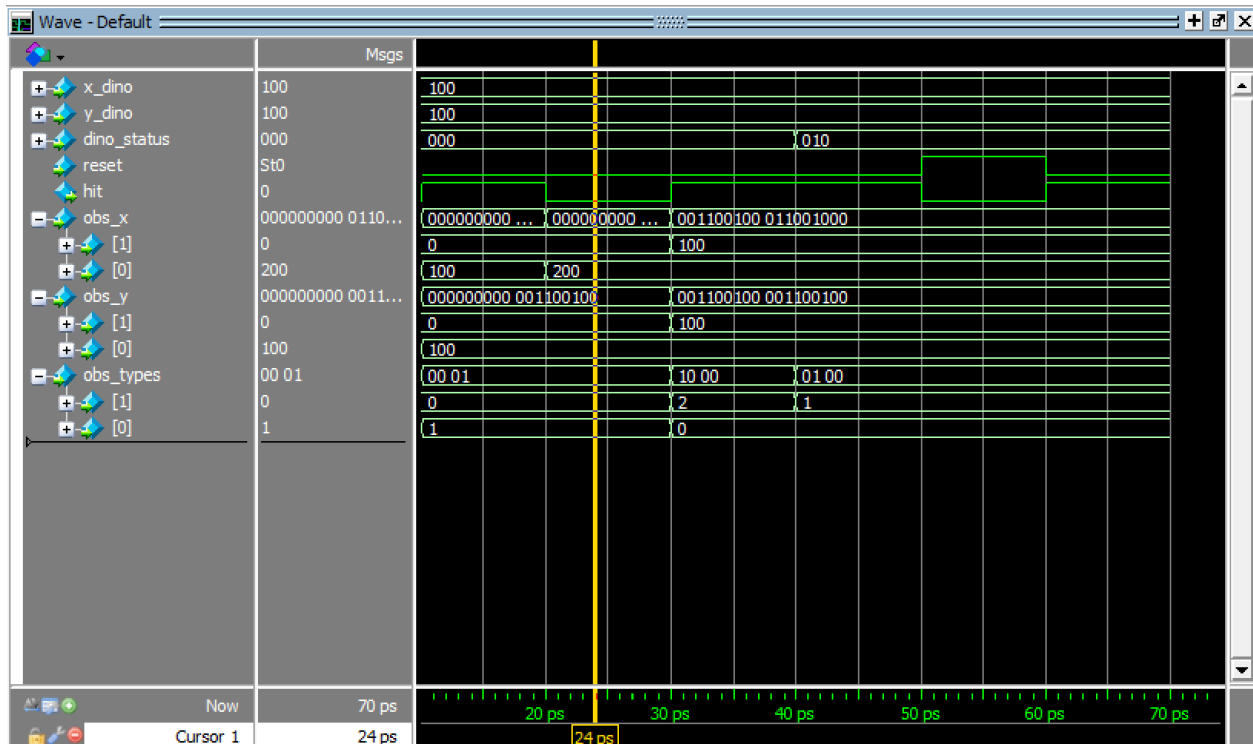
Scoreboard:

The Scoreboard module is a BCD (Binary-coded Decimal) Counter that displays the score that increments over time in decimal to the HEX displays. Each digit will correspond to each of the HEX because we

thought this method provides the best readability and is more similar to how the original Dino game operates. The score freezes upon game over (when “hit” signal goes high) to show the player its current best score.

Results:

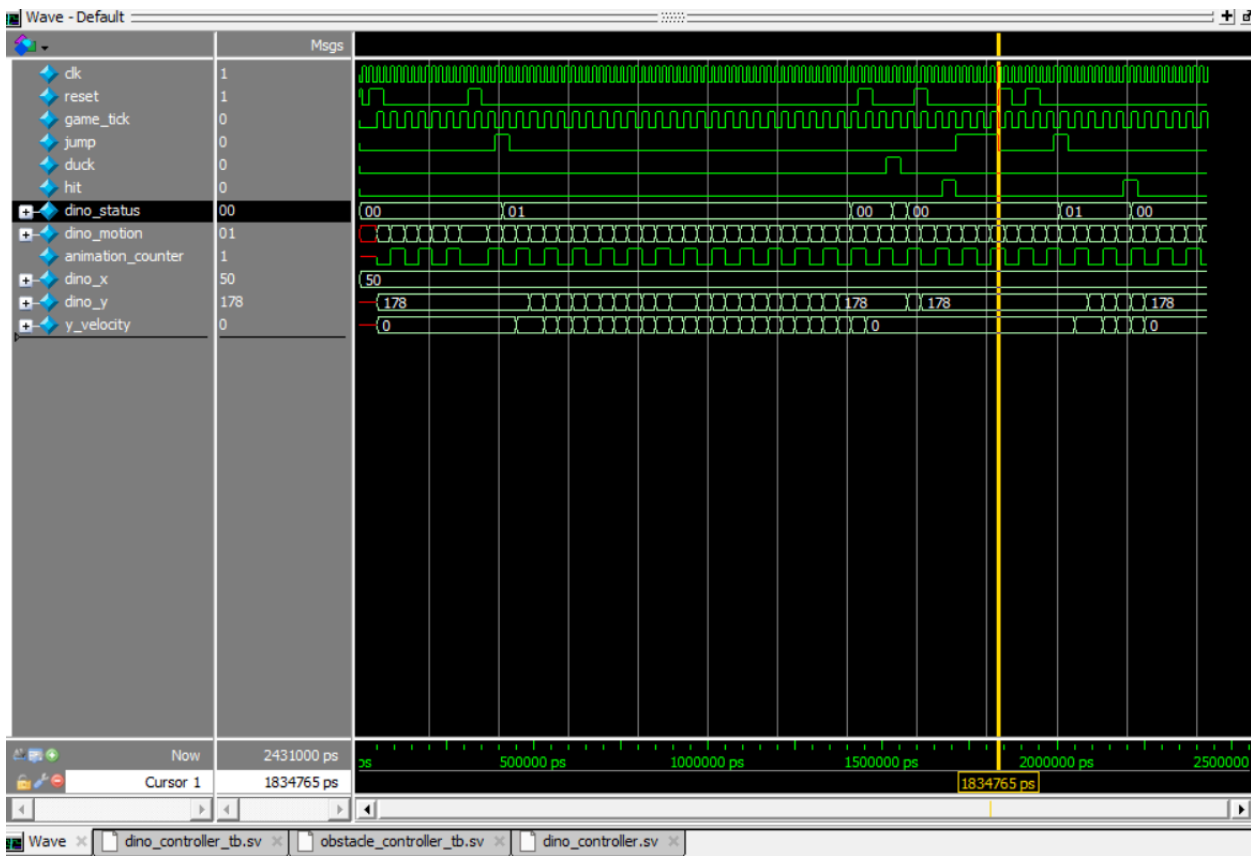
Hit_Box_detector:



```
# Test 1 no obstacle: hit = 0
# Test 2 cactus overlap ( first obstacle): hit = 1
# Test 3 cactus far (first obstacle): hit = 0
# Test 4 pterosaur overlap( second obstacle): hit = 1
# Test 5 duck overlap ( second obstacle): hit = 1
# Test 6 reset: hit = 0
# ** Note: $stop      : ./hit_box_detector_tb.sv(86)
#      Time: 70 ps  Iteration: 0  Instance: /hit box detector tb
```

This is the testbench of the hitbox, it goes over the easy 6 test. For the first test , we set all the obs to 0 which means nothing, and overlap their x, and y with dino_x and y. The result shows there is no hit. As expected. Then we change the first obs to the cactus, since there is overlap, hit = 1; the next test we move the obs far away from the dino’s x and y. The result shows there is no hit. The 4th test we set the first obs to nothing and then set the 2nd obs to pterosaur which is 2, and overlap them, the result shows there is a hit. Next we set dino’s motion to ducking, since the pterosaur didn’t change there is also a hit. And we have a reset test when rest is high the hit is 0 as expected. So overall the hitbox testing is as expected.

Dino_Controller



```

# TEST 1: running
# tick & x=50 y=178 status=0
# tick & x=50 y=178 status=0
# tick & x=50 y=178 status=0
# tick & x=50 y=178 status=0
# tick & x=50 y=178 status=0
# PASS: y=178 (GROUND_Y), status=RUN
#
# TEST 2: jumping
# entered jump & status=1 (expect 1=JUMP)
# airborne: y=178
# airborne: y=178
# airborne: y=168
# airborne: y=158
# airborne: y=149
# airborne: y=141
# airborne: y=134
# airborne: y=128
# airborne: y=123
# airborne: y=119
# airborne: y=116
# airborne: y=114
# airborne: y=113
# airborne: y=113
# airborne: y=114
# airborne: y=116
# airborne: y=119
# airborne: y=123
# airborne: y=128
# airborne: y=134
# airborne: y=141
# airborne: y=149
# airborne: y=158
# airborne: y=168
# airborne: y=178
# landed: y=178 status=0
# PASS: landed at GROUND_Y=178, status=RUN
#

```

TEST 3: ducking

```

duck held:      y=187 status=2 (expect y=187, status=2=DUCK)
duck released: y=178 status=0 (expect y=178, status=0=RUN)
PASS

```

TEST 4: dying

```

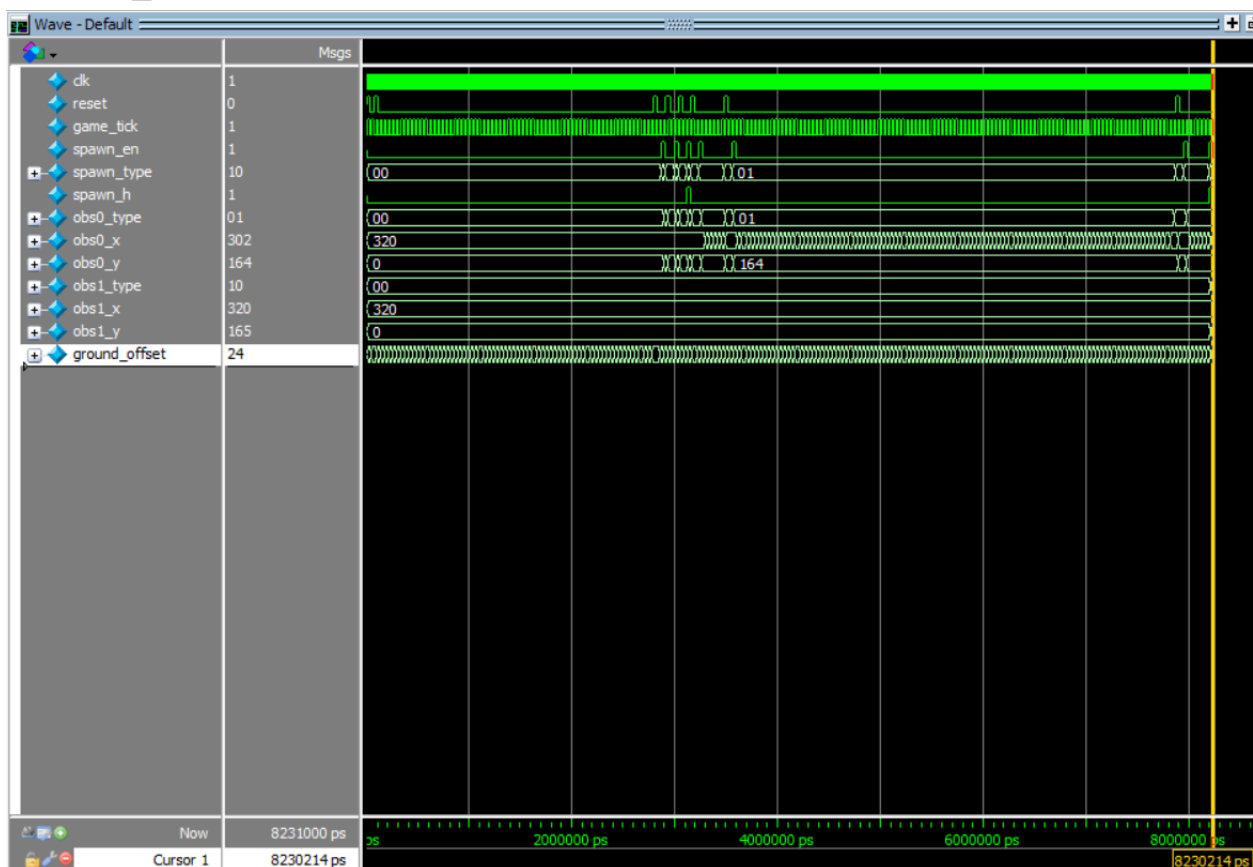
hit applied & y=178 status=0
jump ignored? y=178 status=0
jump ignored? y=178 status=0
jump ignored? y=178 status=0
PASS: jump ignored while dead
after reset & y=178 status=0 (expect y=178, status=0)
PASS: recovered

```

The Dino_Controller tests for each of the motion during the game (running, jumping, ducking, and dead). It begins by checking that the dino remains stationary at the ground level (y=178) with a RUN status

while no inputs are active, passing if both values match exactly. It then verifies jumping behavior by pressing the jump input, confirming the dino transitions into the JUMP state and rises above the ground, and ultimately lands back at $y=178$ with status returned to RUN. A similar check is done for ducking, where holding the duck input should lower the dino to $y=187$ with a DUCK status, and releasing it should bring the dino back to $y=178$ with RUN status, passing only if both the held and released states match their expected values. The dying test asserts the hit signal while the dino is running, expecting the dino's position to freeze in place and ignore further jump inputs, with the only way to recover being a reset that returns it to RUN at $y=178$. Finally, a related test checks that a hit occurring mid-air while the dino is jumping immediately freezes its y position, passing only if y remains constant on every subsequent tick after the hit.

Obstacle_Controller



```
TEST 1: ground offset scroll + wrap
tick & ground_offset=6
tick & ground_offset=9
tick & ground_offset=12
tick & ground_offset=15
tick & ground_offset=18
at offset=198 (expect 198)
after wrapping, the ground_offset=0 (expect 0)
PASS: ground offset wrapped correctly

TEST 2: cactus spawn
after spawn: obs0: type=1 x=320 y=164
                (expect type=1 x=320 y=164)
PASS!

TEST 3: bird low spawn
after spawn & obs0: type=2 x=320 y=172
                (expect type=2 x=320 y=172)
PASS!

TEST 4: bird high spawn
after spawn & obs0: type=2 x=320 y=165
                (expect type=2 x=320 y=165)
PASS!

TEST 5: obstacle scrolling
tick & obs0_x=317 (expect 317)
tick & obs0_x=314 (expect 314)
tick & obs0_x=311 (expect 311)
tick & obs0_x=308 (expect 308)
tick & obs0_x=305 (expect 305)
PASS: scrolled correctly to x=305

TEST 6: obstacle despawn
spawned at x=320, scrolling to despawn...
despawned & obs0: type=0 x=2 (expect type=0)
PASS

TEST 7: two obstacles at once
after 1st spawn & obs0: type=1 x=320 y=164
after 2nd spawn & obs0: type=1 x=302 obs1: type=2 x=320
                (expect obs0=cactus, obs1=bird at x=320)
PASS
TESTING COMPLETE!
```

In Obstacle Controller, the first test verifies that `ground_offset` increases by `SCROLL_SPEED` on every game tick and correctly wraps back to zero once it reaches `GROUND_TEX_W`, passing if the offset equals zero immediately after the wrap. The next three tests check obstacle spawning: spawning a cactus should place `obs0` at `x=320`, `y=164` with `type=1`; spawning a bird with `spawn_h=0` should place it at `y=172` with `type=2`; and spawning a bird with `spawn_h=1` should place it at `y=150` with `type=2`, with each test passing only if all three fields (`type`, `x`, and `y`) match their expected values. The scrolling test spawns a cactus and confirms its `x` position decreases by `SCROLL_SPEED` on every subsequent tick, passing if the position matches the expected decrement after a fixed number of ticks. The despawn test lets a spawned obstacle scroll until its `x` value reaches `SCROLL_SPEED` or below, expecting its type to reset to `NONE`, and passes if `type` equals zero at that point. The final test spawns a cactus, lets it scroll partway across the screen, then spawns a bird, expecting the second obstacle to occupy `obs1` rather than overwrite `obs0`, passing only if `obs0` retains the cactus and `obs1` correctly holds the newly spawned bird at `x=320`.

Flow Summary:

Flow Summary	
 <<Filter>>	
Flow Status	Successful - Wed Jun 10 20:03:44 2026
Quartus Prime Version	17.0.0 Build 595 04/25/2017 SJ Lite Edition
Revision Name	DE1_SoC
Top-level Entity Name	DE1_SoC
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	N/A
Total registers	287
Total pins	118
Total virtual pins	0
Total block memory bits	0
Total DSP Blocks	0
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	1
Total DLLs	0